Cette séance a pour but de vous aider à bien démarrer sur le projet en prenant en main le code fourni et en commençant la conception de votre futur programme.

Vous n'aurez probablement pas le temps de faire tous les exercices pendant les deux heures. Ce n'est pas grave, il n'y a rien à rendre à la fin de la séance. Par contre, il est quand même recommandé de poursuivre ces étapes pour bien démarrer le projet, que ce soit pendant la séance de TP ou après.

Pendant cette séance, votre groupe va devoir :

— comprendre comment le code de départ est organisé (architecture MVC),
— documenter les méthodes et les classes utilisées pendant une frame du jeu,
— dessiner et étendre l'automate à états finis qui gère les états du jeu,
— faire une première ébauche des classes de votre futur programme,
— créer votre dépôt GitHub, faire vos premiers commits, et vous répartir le travail.

Avant de commencer, installez `pygame` (`pip install pygame`) et lancez le jeu.

Exercice 1 : Architecture MVC

Un jeu est un programme particulier, qui doit faire trois choses très différentes : lire ce que font les joueurs (clavier), calculer ce qui se passe (physique, règles de la course), et afficher le résultat à l'écran.

Le patron de conception Modèle Vue Contrôleur (MVC) aide à structurer ce genre de programme en séparant le code en trois partie :

— Modèle (M) : les données et les règles du jeu. Le modèle sait où sont les voitures, à quelle vitesse elles vont, à quoi ressemble le circuit, si un point de passage a été franchi, qui a gagné la course, etc. Par contre, le modèle ne connaît ni le clavier ni l'écran : il ne dessine rien et ne lit aucune touche.
— Vue (V) : l'affichage. La vue lit l'état du modèle et le dessine à l'écran avec `pygame`. Attention, la vue ne modifie jamais le modèle, elle se contente de l'afficher.
— Contrôleur (C) : le lien entre l'utilisateur et le programme. Le contrôleur lit les événements du clavier, les traduit en actions sur le modèle (" le joueur 1 accélère", "le circuit sélectionné change"), et fait tourner la boucle principale du jeu.

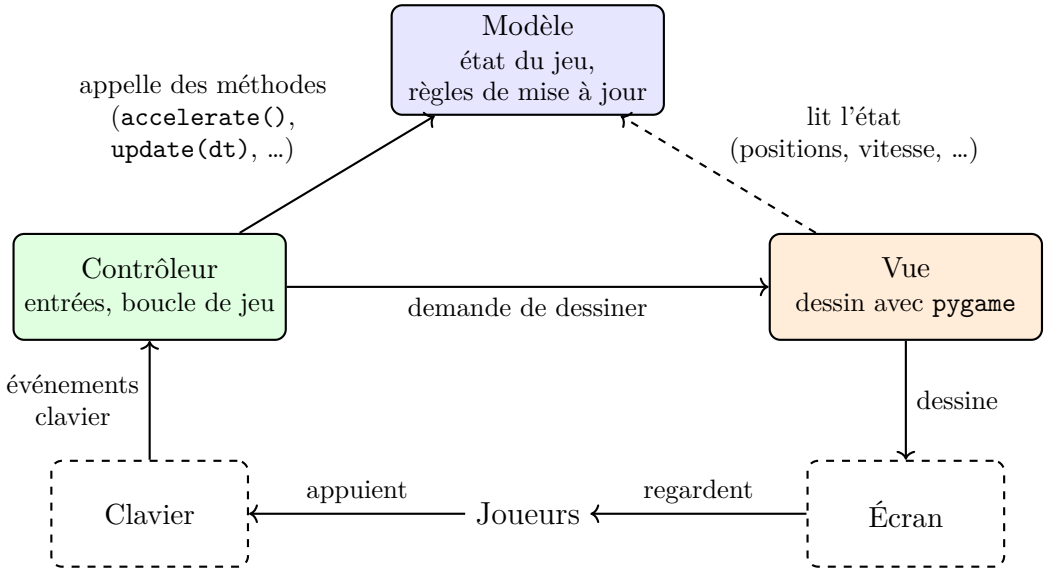

Figure 1 – Les trois parties de l'architecture MVC et la façon dont elles communiquent. La flèche en pointillés signifie que la vue ne fait que lire le modèle.

La boucle principale du jeu ne s'arrête jamais (jusqu'à ce qu'on quitte le jeu) : même si personne ne touche le clavier, les voitures continuent d'avancer et l'écran continue d'être redessiné. Chaque itération de cette boucle s'appelle une frame. Dans le code de départ, il y a 60 frames par seconde (FPS).

Suivre cette architecture a plusieurs intérêts :

— Lisibilité : quand vous cherchez un bug dans la physique de la voiture, vous savez qu'il se trouve dans le modèle, et pas caché dans le code de dessin.
— Indépendance : vous pouvez changer complètement l'apparence du jeu sans toucher une seule ligne de la physique, et inversement.
— Travail en équipe : un membre peut travailler sur la vue pendant qu'un autre travaille sur le modèle, à condition qu'ils se mettent d'accord sur les attributs et les méthodes que fournit le modèle. Attention

toutefois, pour l'évaluation vous pourrez être interrogés sur toutes les parties. Comme les concepts de programmation orientée objet vont surtout jouer un rôle dans la partie modèle, il est recommandé que chacun participe au modèle.

— Tests : le modèle peut être testé sans ouvrir de fenêtre. Vous pouvez par exemple écrire des tests non graphiques pour vérifier que votre modèle a le comportement désiré lorsqu'il reçoit certaines entrées.

Bien que ce soit recommandé, il n'est pas obligatoire de suivre à la lettre cette architecture pour le projet. D'ailleurs, le code de départ inclut l'affichage directement dans la classe `RoadTile`, ce qui n'est pas parfaitement en accord avec l'architecture MVC.

1. Passez quelques minutes à comprendre l'organisation du code.
2. Si vous voulez implémentez le freinage et la marche arrière, allez vous modifier la partie modèle, vue et/ou contrôleur ? Quels fichiers vont être modifiés ?
3. Implémentez le freinage et la marche arrière.

Exercice 2 : Suivre l'exécution d'une frame

Tracer l'exécution d'un programme consiste à suivre son déroulement pas à pas, appel de méthode après appel de méthode, pour comprendre ce qu'il fait. C'est une manière efficace de comprendre du code écrit par quelqu'un d'autre, et une compétence dont vous aurez besoin pendant l'examen oral, où vous pourrez être interrogés sur n'importe quelle partie du code.

1. En partant du début de la boucle de jeu, listez dans l'ordre tous les appels de méthodes effectués pendant une frame d'une course où le joueur maintient la touche "avancer". Pour chaque appel, notez à quelle partie du MVC il appartient et ce qu'il fait. Il est utile de garder une trace de quelle méthode est appelée depuis quelle autre méthode.
2. Écrivez une docstring pour chaque classe et chaque méthode qui apparaît dans votre liste, et ajoutez des commentaires partout où c'est utile pour la compréhension.

Exercice 3 : Machines à états

Une machine à état (ou automate) est une manière simple de décrire un système qui peut se comporter de différentes manières pour les mêmes inputs selon l'état dans lequel il est. Elle est composée :

— d'un ensemble fini d'états : les situations possibles du système.

— d'un état initial : l'état du système au démarrage.

— de transitions : des flèches d'un état vers un autre, chacune étiquetée par l'événement qui la déclenche.

À tout moment, le système se trouve dans exactement un état, appelé l'état courant. Quand un événement se produit (par exemple le joueur ou la joueuse appuie sur le clavier), on cherche une transition qui part de l'état courant avec cet événement : s'il y en a une, l'état courant change et s'il n'y en a pas, il ne se passe rien.

Par convention, un état est représenté par un cercle contenant son nom, une transition par une flèche étiquetée par son événement, et l'état initial est indiqué par une flèche entrante qui ne part d'aucun état.

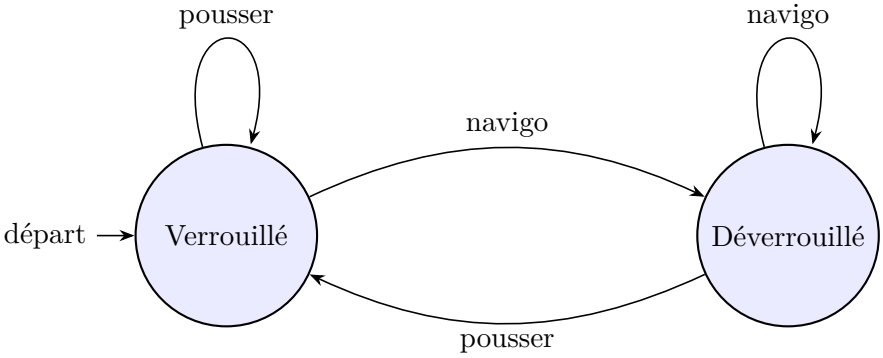

Figure 2 – Automate d'un tourniquet.

Exemple : un tourniquet de métro. Un tourniquet est verrouillé au départ. Passer votre passe navigo le déverrouille. Le pousser quand il est déverrouillé laisse passer une personne et le verrouille à nouveau. Pousser un

tourniquet verrouillé, ou passer son passe navigo sur un tourniquet déverrouillé, ne change rien (on le représente par des flèches qui bouclent sur le même état).

Les écrans d'un jeu (menu principal, sélection du circuit, course, pause, …) se décrivent naturellement par un automate. L'état courant détermine :

- Comment le contrôleur interprète les touches : la touche Entrée valide un choix dans un menu, mais utilise un objet pendant une course.
- Ce que le modèle met à jour : les voitures ne se déplacent que pendant la course.
- Ce que la vue dessine : un menu ou le circuit.

En Python, l'état courant est généralement stocké dans un attribut, et les transitions sont écrites sous forme de conditions sur cet attribut.

1. Trouvez où l'état du jeu est stocké et modifié dans le code de départ. Dessinez l'automate du code de départ : ses états, son état initial, et ses transitions étiquetées par les événements (touches) qui les déclenchent.
2. On veut maintenant que les joueurs choisissent le circuit avant la course.
   - Dessinez le nouvel automate. Quel(s) état(s) ajoutez-vous ? Quelles transitions ? Comment un joueur revient-il au menu principal ?
   - Créez un deuxième circuit (il peut être très simple), et implémentez la sélection du circuit. La vue doit afficher la liste des circuits et mettre en évidence celui qui est sélectionné.
   - Vérifiez que votre implémentation correspond à votre dessin : essayez chaque touche dans chaque état.
3. (Facultatif) On a proposé d'utilisé les automates pour les écrans du jeu, mais on pourrait aussi imaginer les utiliser pour gérer l'état de la voiture. Dessinez l'automate d'une voiture qui peut soit adhérer à la route, soit déraper : quels événements ou quelles conditions la font commencer et arrêter de déraper ?

Exercice 4 : Concevoir les classes de votre programme

Avant d'écrire la suite du programme, vous devez décider quelles classes il contiendra. Le résultat de cette section est une première version de votre diagramme de classes (qui va probablement évoluer au cours du projet).

1. Relisez le sujet du projet, et listez tous les noms qui pourraient correspondre à une classe de votre programme (par exemple : voiture, terrain, …).

   Pour chaque nom, décidez s'il doit vraiment être une classe, ou plutôt un attribut d'une autre classe. Un nom qui possède ses propres données et son propre comportement est un bon candidat pour être une classe. A l'inverse, un nom qui n'est qu'une valeur (une vitesse, un nombre de tours) est plutôt un attribut. Pensez aussi à inclure les classes de la vue et du contrôleur.
2. Pour chaque classe retenue dans la question précédente, listez ses principaux attributs (avec leurs types) et ses principales méthodes (avec les arguments et les types de sortie).
3. (ROB, SI, MEDH uniquement)
   - Trouvez les groupes de classes qui partagent des attributs ou des méthodes. Pour chaque groupe, proposez une classe parente. Lesquelles de ces classes parentes devraient être abstraites, et quelles méthodes devraient être abstraites ? Pour rappel, votre programme doit utiliser l'héritage avec au moins deux classes parentes différentes, et au moins une classe abstraite.
   - Identifiez les autres relations entre vos classes (composition, agrégation, association). Par exemple : de quoi est composé un circuit ? Que contient une case du circuit ?
4. En vous basant sur les réponses précédentes, dessinez la première version de votre diagramme de classes UML.

Exercice 5 : Dépôt et organisation

Votre implication dans le projet sera évaluée à travers votre dépôt GitHub : il doit être créé dès maintenant, et chaque membre du groupe doit y contribuer régulièrement.

1. Pour commencer, choisissez un membre du groupe qui doit :
   - Créer un dépôt GitHub public et ajoute les autres membres comme collaborateurs.
   - Ajouter le code de départ et un fichier `requirements.txt`.
   - Créer un fichier `README.md` avec les instructions pour installer et lancer le jeu, et une section vide pour la déclaration de votre utilisation des outils d'IA, que vous compléterez au cours du projet.

2. Chaque membre du groupe clone le dépôt et fait au moins un commit : par exemple, les docstrings et commentaires de l'exercice 2 peuvent être répartis entre les membres. Pensez à utiliser des messages de commit informatifs. Par exemple "update" et "fix" ne sont pas des noms de commits assez précis. Préférez "Ajoute le freinage et la marche arrière à la classe Car" et "Corrige la direction de virage de la voiture". Pensez à faire un `pull` avant de commencer à travailler, et à faire souvent des commits de petites modifications cohérentes plutôt qu'un énorme commit à la fin.
3. À partir de votre diagramme de classes, répartissez le travail entre les membres du groupe :
    1. Attribuez les classes (ou les fonctionnalités) aux membres, de sorte que chacun travaille sur une partie significative du programme, y compris le modèle.
    2. Pour les classes qui interagissent entre elles mais sont écrites par des membres différents, mettez-vous d'accord dès maintenant sur les noms et les paramètres des méthodes qui seront appelées de l'autre côté.
    3. Planifiez votre travail jusqu'à la date limite, avec des objectifs intermédiaires. Écrire la répartition des tâches et le planning dans le README de votre dépôt aide tout le monde à s'y retrouver.

Pour suivre votre progression

- ☐ La voiture peut freiner et reculer.
- ☐ Les méthodes appelées pendant une frame sont documentées.
- ☐ L'automate du jeu est dessiné, et la sélection du circuit fonctionne.
- ☐ La première version du diagramme de classes est dessinée.
- ☐ Le dépôt existe, et chaque membre a fait au moins un commit.
- ☐ Le travail est réparti, et un planning est écrit.